009

Virtual Address Translation and Page Tables

Operating Systems

Virtual Address Translation and Page Tables

Objective

You will implement a **program** (simulation, normal user-space process) that models **paged address translation** for **one** loaded process: **split virtual addresses** into **VPN** and **offset**, keep a **linear page table**, allocate **physical frames** from a **fixed-size RAM**, and **translate a list of virtual addresses** to **physical addresses** or print **clear errors** when translation is impossible.

Scenario: Where the bytes really live

An OS has already chosen a **page size** and a **virtual address layout**. A process is **loaded**: each of its **virtual pages** receives a **physical frame** from free memory. Later, the loader—or a debugger—feeds **virtual addresses** one by one. For each address your program must answer: **which physical address** corresponds to that access, or **why** the access is invalid.

This lab only practices **decomposition + page table + frame allocator + translation**.

Assignment Tasks

Task 1 — Virtual address format (given; do not change)

Everyone uses **the same** layout so outputs are comparable.

Field	Bits	Meaning
Offset	**[7 : 0]**	Byte index inside the page (**8** bits).
VPN	**[15 : 8]**	Virtual page number (**8** bits).

Constants:

- Virtual address: **unsigned 16-bit**, range **0x0000 .. 0xFFFF**.
- **Page size / frame size: 256** bytes.

Extract:

- offset = virtual_address & 0xFF
- vpn = (virtual_address >> 8) & 0xFF

Checks:

- If an input value is **> 0xFFFF**, reject with error **VA_OUT_OF_RANGE**.
- The process has **NUM_VIRTUAL_PAGES** pages, with **1 ≤ NUM_VIRTUAL_PAGES ≤ 256**. Only **vpn < NUM_VIRTUAL_PAGES** may use the page table. If **vpn ≥ NUM_VIRTUAL_PAGES**, error **VPN_OUT_OF_RANGE**.

Task 2 — Parse and display

Write a routine that, for each virtual address:

1. Computes **vpn** and **offset** using Task 1.
2. Prints them in hex (or decimal, but be **consistent** with your sample output section below).

Task 3 — "Imagined RAM", free frames, load, page table

Physical memory

- **NUM_FRAMES = 100** frames, indices **0 .. 99**.
- Each frame: state **FREE** or **OCCUPIED**.
- **Initial RAM (random):** before loading the process, **randomly** mark a subset of frames **OCCUPIED** (simulate "already in use" holes). Rules:
    - **OCCUPIED** count: **random** in **[10, 60]** (or another interval you document) so the map is nontrivial but leaves room to load.
    - After marking, **FREE** count must be **≥ max(10, V)** where **V = NUM_VIRTUAL_PAGES** for this run (so load can succeed). If your first random draw fails that check, **repeat** the randomization until it passes (cap iterations in code and document).
    - Use a **pseudo-random** source. **Optional CLI seed** for reproducible maps while debugging; otherwise document default (e.g. time-based).

Memory printout (required)

Right after RAM randomization (and **before** process load), print a **visible map** of physical memory so anyone can see which frames are taken. At minimum:

- One line summary: **FREE** count / **OCCUPIED** count.
- A **per-frame listing** for **0 .. 99**: each line or column shows **frame index** and **F** (free) vs **X** (occupied), **or** a compact block (e.g. 10 frames per row). Example shape (your numbers will differ):

```
PHYSICAL RAM (100 frames) after random init (seed=12345):
FREE=82 OCCUPIED=18
```

0:X 1:F 2:X 3:F 4:F 5:X 6:F 7:F 8:F 9:X
10:F 11:F …

Allocator

- **allocate_frame()**: returns one **FREE** frame index, marks it **OCCUPIED**. If none free, signal failure.

Page table

- One **linear** page table with **NUM_VIRTUAL_PAGES** entries.
- Each entry (**PTE**): at least **valid** (boolean) and **pfn** (frame index when valid).

Load process

- Input parameter: **NUM_VIRTUAL_PAGES** (call it **V** in the CLI).
- For **i = 0 .. V-1**: allocate a frame; on success set **PTE[i].valid = true**, **PTE[i].pfn =** allocated index.
- If any allocation fails mid-load, **abort** with a clear message and leave the table in a state you document (all invalid, or rollback).

Sanity: before load, assert **V ≤ FREE_count** (guaranteed by the randomization rule above).

Task 4 — Batch translation

- Read a **text file**: **one unsigned virtual address per line** (decimal or **0x** hex—**state which you support**).
- For each line: translate using the page table and **PA = PTE[vpn].pfn * 256 + offset** when valid.
- If **PTE[vpn].valid is false**, error **PAGE_NOT_MAPPED**.
- Print **one line per address** (success or error). See **Sample output** below for the expected shape.

Reference dataset (use this in your submitted log)

Use these **exact** translation inputs; **physical frame numbers and PAs** will differ every run because RAM is **random**—grading checks **self-consistency** (VPN/offset math, PTE lookup, PA = PFN×256+offset, and errors), not a single golden PFN list.

Setting	Value
NUM_VIRTUAL_PAGES	**8** (VPNs **0 .. 7** valid)
RAM init	**Random** occupied pattern per Task 3, then **print the map**.
Address list file (decimal, one per line)	Same as **example_addresses.txt: 0, 255, 256, 512, 3840, 70000**

Expected **logical** outcomes (must match in every run):

- **0** → VPN **0**, offset **0** → success if **PTE[0]** valid (**PFN** from **your** run).
- **255** → VPN **0**, offset **255** → success if **PTE[0]** valid.
- **256** → VPN **1**, offset **0** → success if **PTE[1]** valid.
- **512** → VPN **2**, offset **0** → success if **PTE[2]** valid.
- **3840** → VPN **15**, offset **0** → with **V = 8** → **VPN_OUT_OF_RANGE**.
- **70000** → **> 0xFFFF** → **VA_OUT_OF_RANGE**.

Sample output

Your labels can vary slightly, but every run must show **(1)** RAM map, **(2)** load summary, **(3)** one line per translated address.

```
PHYSICAL RAM (100 frames) after random init (seed=12345):
FREE=82 OCCUPIED=18
 0:X  1:F  2:X  3:F  4:F  5:X  6:F  7:F  8:F  9:X
... (all frames 0..99 shown) ...

Load process: V=8 -> VPN 0..7 mapped to PFNs [ .. from your allocator on the FREE set .. ]

VA=0x0000 (0)    VPN=0x00 OFF=0x00  PFN=<from PTE0>  PA=<PFN*256+0>
VA=0x00FF (255)  VPN=0x00 OFF=0xFF  PFN=<same as PTE0>  PA=<PFN*256+0xFF>
VA=0x0100 (256)  VPN=0x01 OFF=0x00  PFN=<from PTE1>  PA=...
VA=0x0200 (512)  VPN=0x02 OFF=0x00  PFN=<from PTE2>  PA=...
VA=3840 (0x0F00)  ERROR=VPN_OUT_OF_RANGE (vpn=15, V=8)
VA=70000        ERROR=VA_OUT_OF_RANGE
```

PFNs and PAs are not fixed: they must agree with **your** printed RAM map and the order **allocate_frame()** picked free frames.

Extra points (optional)

- **2-D or color** RAM visualization (still 100 frames).
- **Second process** (second page table) sharing the same RAM pool (document frame ownership).